



INTERNATIONAL
HELLENIC
UNIVERSITY

International Hellenic University

School of Science and Technology

**A Mathematical and Practical Study of the
PRESENT Cipher**

Lazaros Papakostas

Course: Data Protection and Cryptography

Professor: Dr.Stavrinides

Contents

Abstract.....	2
Introduction.....	3
What is the PRESENT Cipher & How It Works.....	3
Key Scheduling Process	7
Decryption in the PRESENT Cipher.....	8
Mathematical Foundations	8
Example (1 Round)	10
Bibliography.....	12

Abstract

This coursework named “A Mathematical and Practical Study of the PRESENT Cipher” delves into the PRESENT cipher, a lightweight block cipher, specifically designed for constrained environments such as IoT devices, RFID tags, and embedded systems. With the growing demand for efficient cryptographic solutions in low-resource platforms, lightweight cryptography has become a crucial and interesting field of study. The PRESENT cipher, standardized by ISO/IEC 29192-2, offers strong security with a compact structure based on a Substitution–Permutation Network (SPN) model.

This paper investigates the mathematical foundations of the cipher with a great focus on the role of the S-box in ensuring non-linearity, the permutation layer's function in achieving diffusion, and the significance of modular arithmetic and bitwise operations in the key scheduling process. A step-by-step one-round encryption example is presented using the official S-box and permutation functions to depict how the cipher transforms plaintext into ciphertext.

Introduction

Traditional cryptographic algorithms like AES and SHA are robust but at the same time they can be too demanding for low-power devices that are found in IoT ecosystems. To address this challenge lightweight cryptography was developed.

Lightweight cryptography or lightweight encryption as it is referred to by many, is a specialized form of encryption, it is developed for devices with limited resources. These algorithms are designed to function with minimal computational power, memory and energy. They can offer secure communication for devices that are operating under strict hardware constraints.

Most lightweight cryptographic systems use symmetric encryption, in which both the sender and the receiver use the same key to encrypt and decrypt messages. This approach reduces computational overhead compared to asymmetric cryptography and allows for faster and energy efficient processing.

The demand for lightweight cryptography has been growing very rapidly over the years. Billions of IoT devices are operating across all industries and sectors. It is a critical operation to secure the massive volume of sensitive data they generate. Futurex. (2023). *How will lightweight cryptography impact you?* Futurex.

One of the most well-known lightweight block ciphers is PRESENT. It is a standardized algorithm, specifically designed for low-resource environments. It is known for its compact design, simple structure and strong security properties, making it suitable for implementation in embedded systems, like RFID tags and other IoT devices. The PRESENT cipher serves as a foundational example of how lightweight cryptography can be applied in practice to secure modern digital infrastructures.

The purpose of this coursework is to explore the PRESENT cipher, examining how it works, where it is used and what are the mathematical principles behind its design. The focus will be on understanding its internal structure, including the use of substitution-permutation networks, S-boxes and bitwise operations, as well as demonstrating how these mathematical components contribute to the cipher's overall security and efficiency.

What is the PRESENT Cipher & How It Works

The PRESENT lightweight block cipher was introduced in 2007 as a joint effort by the Orange Labs (based in France), Ruhr University of Bochum and the Technical University of Denmark. The algorithm is also known for its compact size, it is about 2.5 times smaller than the AES.

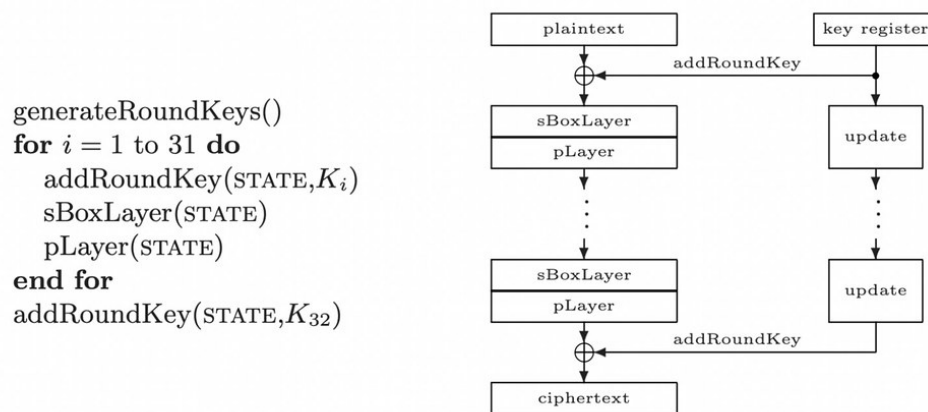
Its structure follows the Substitution-Permutation Network (SPN) model. It has a 64-bits block size, and the key size can be either 80 bits or 128 bits and consists of 31 rounds. The non-linear layer is based on a single 4-bit S-box, which was designed

with hardware optimizations in mind. PRESENT is intended to be used in situations where low-power consumption and high chip efficiency are desired. Bogdanov, A., Knudsen, L. R., Leander, G., Paar, C., Poschmann, A., Robshaw, M. J., ... & Vikkelsoe, C. (2007)

Due to these features, PRESENT has been standardized in ISO/IEC 29192-2, which defines lightweight block ciphers for security in constrained environments.

PRESENT is designed with bit-level permutations, which makes it less suitable for software-based implementations. Its architecture is clearly optimized for hardware environments, where such bit-permutations can be efficiently handled through simple wiring. When evaluated in software, particularly on microcontrollers, the performance of PRESENT has been assessed using FELICS (Fair Evaluation of Lightweight Cryptographic Systems), a benchmarking framework specifically developed to measure the efficiency of lightweight cryptographic algorithms in software environments. (Wikipedia)

Figure 1



Explaining the three functions

As we can see from “Figure 1”, each round in the PRESENT Cipher consists of three distinct functions

- **addRoundKey()**

The addRoundKey() function involves performing an XOR between the output of the previous round (or the initial plaintext in the first round) and the round-specific key.

This round key is derived from the master key using a process called key scheduling, which generates a unique subkey for each round.

For round i , round key $k^i = k_{63}^i \dots k_0^i$ is XORed with the data b_j

$$b_j \rightarrow b_j \oplus k_j^i$$

- **sBoxLayer()**

The sBoxLayer() function breaks the 64-bit data block into sixteen 4-bit segments and applies substitution using 16 identical S-boxes. Each 4-bit segment is replaced with another 4-bit value, based on a predefined substitution table.

Table 1

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S[x]	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

As we can see from **Table 1** The S-box used in PRESENT maps each 4-bit hexadecimal input to a unique 4-bit hexadecimal output. This substitution introduces non-linearity into the cipher, which is essential for creating confusion, a key cryptographic property.

As shown in the table, every input value has a distinct output, and there is no case where the input equals the output. Additionally, changing just one bit in the input leads to multiple bits changing in the output, strengthening resistance against attacks.

- **pLayer()**

The pLayer function is responsible for rearranging the bits of the state (data block) using a fixed permutation table. Each bit at position i is moved to a new position $P(i)$ according to Table 2.

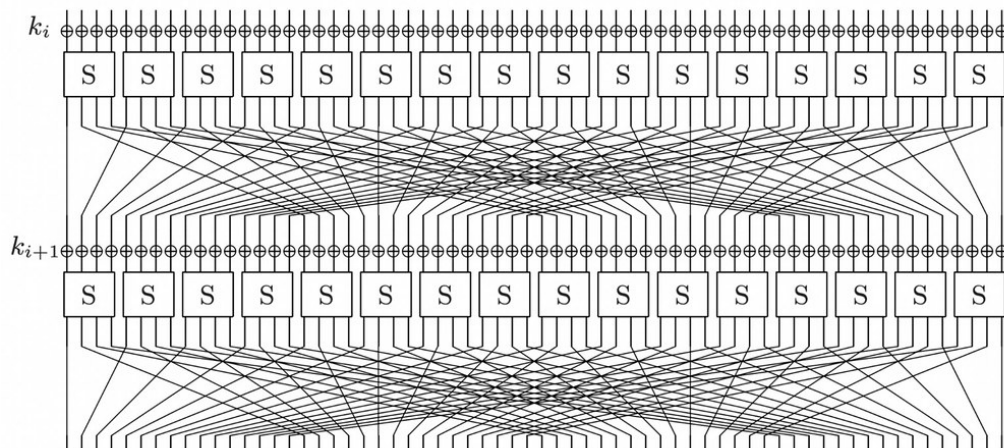
Table 2

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P[i]	0	16	32	48	1	17	33	49	2	18	34	50	3	19	35	51
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
P[i]	4	20	36	52	5	21	37	53	6	22	38	54	7	23	39	55
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
P[i]	8	24	40	56	9	25	41	57	10	26	42	58	11	27	43	59
i	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
P[i]	12	28	44	60	13	29	45	61	14	30	46	62	15	31	47	63

The sBoxLayer() function (substitution layer) by itself does not achieve sufficient diffusion, since each 4-bit input only influences its corresponding 4-bit output. To enhance diffusion, the cipher includes a permutation layer. This layer ensures that bits are spread across the entire block.

As we can see from Figure 2, while the leftmost and rightmost bits remain in their original positions, all other bits are redistributed along unique and distinct paths.

Figure 2



These three functions are repeated for each round. When all 31 rounds are completed, the output is XORed with the round key for the purpose of key whitening, which adds an extra layer of security to the ciphertext.

Key Scheduling Process

The PRESENT cipher supports both 80-bit and 128-bit keys. In this presentation, we focus on the key schedule mechanism for the 80-bit version.

From the 80-bit master key, in each encryption round a 64-bit is derived from it. This master key is stored in a key register. At the end of each round, the key consists of the leftmost 64 bits of the current state of this register. Bogdanov, A., Knudsen, L. R., Leander, G., Paar, C., Poschmann, A., Robshaw, M. J., ... & Vikkelsoe, C. (2007)

For example, during the first round, the initial 64 bits from the master key are directly used as the round key. After each round, the key register follows through the next steps:

1st step: Left rotate the 80-bit register by 61 positions

$$K_{79}K_{78} \dots K_1K_0 = K_{18}K_{17} \dots K_{20}K_{19}$$

2nd step: Pass the leftmost 4 bits through the PRESENT S-box for substitution.

$$K_{79}K_{78}K_{77}K_{76} = S[K_{79}K_{78}K_{77}K_{76}]$$

3rd step: XOR bits (19,18,17,16,15) of the register with the current round number, known as the round counter

$$K_{19}K_{18}K_{17}K_{16}K_{15} = [K_{19}K_{18}K_{17}K_{16}K_{15}] \oplus \text{roundcounter}$$

Decryption in the PRESENT Cipher

In the PRESENT cipher the encryption and decryption process have nearly identical requirements. The decryption process involves reversing certain operations performed during encryption. The key schedule remains the same for both encryption and decryption process

During the decryption process the ciphertext is used as an input and following steps are applied in reverse

1st step Inverse Permutation Layer (InverseP-Layer):

This step undoes the bit-level permutations applied during encryption, restoring the original bit positions.

2nd step Inverse S-box Layer:

The state is processed through the inverse of the S-box, reversing the non-linear substitution performed during encryption.

3rd step AddRoundKey:

The current state is XORed with the corresponding round key, just like in encryption. This operation is its own inverse.

After all rounds have been reversed, the original plaintext is recovered. Bogdanov, A., Knudsen, L. R., Leander, G., Paar, C., Poschmann, A., Robshaw, M. J., ... & Vikkelsoe, C. (2007)

Mathematical Foundations

A Substitution-Permutation Network (SPN) is a series of linked mathematical operations. The PRESENT block cipher is built upon an SPN model, which combines non-linear substitution and bit-level permutation operations across multiple rounds. Bogdanov, A., Knudsen, L. R., Leander, G., Paar, C., Poschmann, A., Robshaw, M. J., ... & Vikkelsoe, C. (2007)

The mathematical expression $R_i(X) = P(S(X \oplus K_i))$ describes each round of the cipher.

Where:

- X is the 64-bit input state to round i
- K_i is the 64-bit round key for round i
- $\oplus$ denotes bitwise XOR
- S is the S-box substitution function
- P is the permutation layer

This mathematical formula ensures that even a single-bit change in the plaintext or key propagates through multiple rounds, affecting many output bits. This is called the avalanche effect. The use of substitution leads to confusion, while the permutation layer introduces diffusion. Both are essential for strong cryptographic security.

The **S-box** (substitution box) is a non-linear Boolean function mapping a 4-bit input to a 4-bit output:

$$S: \{0,1\}^4 \rightarrow \{0,1\}^4$$

PRESENT uses 16 parallel S-boxes per round, each applying the same substitution to a different 4-bit segment of the 64-bit state. The mathematical properties of the S-box are critical to the cipher's security:

- Non-linearity: Measures how much the S-box differs from all affine functions.
- Differential Uniformity: Determines how uniformly output differences are distributed for given input differences
- Avalanche Effect: A small change in input results in a significant change in output.

These properties are carefully chosen during the design of the PRESENT S-box to maximize cryptographic strength while keeping hardware implementation efficient.

Dey, S., & Ghosh, R. (2018).

The **P-layer** ensures bit diffusion by reordering the positions of the 64 bits in the state according to a fixed permutation table. Each bit is moved to a new location using the formula:

$$P(i)=16 \cdot i \bmod 63, \text{ for } 0 \leq i \leq 62, P(63)=63$$

This operation ensures that the output of each S-box in one round is distributed across multiple S-boxes in the next round, enhancing diffusion across rounds. Bogdanov, A., Knudsen, L. R., Leander, G., Paar, C., Poschmann, A., Robshaw, M. J., ... & Vikkelsoe, C. (2007)

The AddRoundKey step uses the bitwise exclusive OR (XOR) operation between the current state and the round key:

$$X \leftarrow X \oplus K_i$$

This operation is linear in the field $\mathbf{F}_2$ and is both efficient and reversible, making it ideal for key mixing in both encryption and decryption. XOR is used in every round of PRESENT, as well as in the final key whitening step after the 31st round. Defend the Web. (n.d.). *The XOR cipher*.

The **key scheduling** of PRESENT involves several operations that rely on modular arithmetic:

- A 61-bit left rotation of the 80-bit or 128-bit master key, which is effectively a modular shift.
- An S-box substitution applied to the most significant 4 bits of the rotated key.
- An XOR of the round counter into bits of the key (using the least significant 5 bits of the round number).

These operations generate a new round key for each round, ensuring round independence and resisting related-key attacks. The modular arithmetic used in these transformations allows efficient cycling of the key while maintaining unpredictability.

Example (1 Round)

To demonstrate how the cipher works we will run only 1 round

We will use:

- A 64-bit block
- A 64-bit round key derived from an 80 bits master key
- PRESENT S-Box
- P-layer permutation

Initial Setup

- Plaintext block: 0x0000000000000000
- Round key: 0xFFFFFFFFFFFFFFFF

Step 1: AddRoundKey (XOR)

XOR the plaintext with the round key:

- Plaintext block: 0x0000000000000000
- Round key: 0xFFFFFFFFFFFFFFFF
- XOR Result: 0xFFFFFFFFFFFFFFFF

Step 2: sBoxLayer

Split the 64-bit block into 16 nibbles (4 bits each):

F F F F F F F F F F F F F F F F

Now we apply the PRESENT S-box:

Input Output

F (15) 2

So all the Fs become 2s:

2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2

sBoxLayer()=0x2222222222222222

Step 3: pLayer

The Permutation Layer repositions each bit according to the rule:

$$P(i)=16 \cdot i \bmod 63, \text{ for } 0 \leq i \leq 62, P(63)=63$$

So:

- Bit 0 $\rightarrow$ Bit 0
- Bit 1 $\rightarrow$ Bit 16
- Bit 2 $\rightarrow$ Bit 32
- Bit 3 $\rightarrow$ Bit 48
- Bit 4 $\rightarrow$ Bit 1
- ...
- Bit 63 $\rightarrow$ Bit 63

The bit at position i moves to position $P(i)$. If you apply this permutation to the binary representation of 0x2222222222222222, the bits get scattered in a specific, fixed pattern across the 64-bit output.

After verification using the P-Layer, the result is: 0x0888888888888888

Final Output after 1-Round:

- Input: 0x0000000000000000
- Key: 0xFFFFFFFFFFFFFFFF
- Output: 0x0888888888888888

Bibliography

Bogdanov, A., Knudsen, L. R., Leander, G., Paar, C., Poschmann, A., Robshaw, M. J. B., Seurin, Y., & Vikkelsoe, C. (2007). *PRESENT: An ultra-lightweight block cipher*. In P. Paillier & I. Verbauwhede (Eds.), **CHES 2007: Cryptographic Hardware and Embedded Systems** (pp. 450–466). Springer.

Defend the Web. (n.d.). *The XOR cipher*.

Dey, S., & Ghosh, R. (2018). *A Review of Existing 4-Bit Crypto S-Box Cryptanalysis Techniques and Two New Techniques with 4-Bit Boolean Functions for Cryptanalysis of 4-Bit Crypto S-Boxes*. *Advances in Pure Mathematics*, 8(3), 272–306.

Wikipedia contributors. (n.d.). *PRESENT (cipher)*.